# M8 LED Animation and Physical Board — Module Proposal

Nov 1, 2026 · @Taha

## 1. Summary and scope

M8 builds the physical 60 × 60 cm city board and writes the animation engine that turns the game state into light. The board has 344 addressable LEDs (336 on the roads, 8 in the four stores), 3D-printed buildings, and mounts for the sensor and motors. The animation engine runs on the board's ESP32 and receives the state from M7 through a small C++ interface.

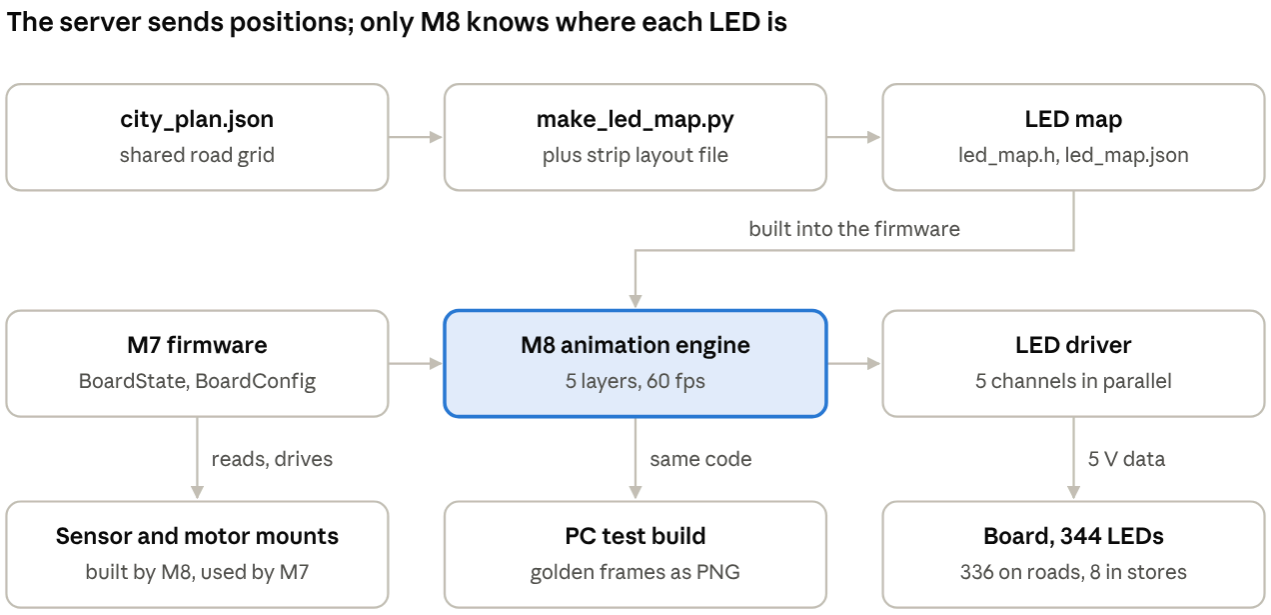
| Item | Detail |
|---|---|
| Members | Ayşe Poyraz, Hasan Hüseyin Özdemir, Muhib Taha Boy, Umutcan Ocak |
| Runs on | ESP32 core 1 (software); the board itself (hardware) |
| In scope | Base, roads, LED strips, wiring, power, 3D-printed buildings and stores, motor and sensor mounts, LED map, all light animations, brightness and power limit |
| Out of scope | Wi-Fi and protocol, sensor reading, motor control (M7); game rules (M1) |

Three design rules keep M8 independent and stable:

- **Positions, not LED numbers.** The server sends city positions; only M8 knows which LED is where. The LED count or wiring can change without any change in other modules.
- **Portable animation code.** The engine is plain C++ with no Arduino or Wi-Fi code, so it also builds on a PC for tests and frame previews.
- **Built to be hit.** People will slap the board for earthquakes, so LEDs sit under protective diffusers and buildings are screwed or pinned, not only glued.

# 2. Position in the system

M8 has a single run-time neighbour, M7, which hands it the latest board state on the same ESP32. Everything else reaches M8 through the shared city plan, which is turned into the LED map at build time.




M8 in the system · build-time LED map and run-time frame path

The top row runs once per city plan version; the middle row runs 60 times a second. Because the same engine code also builds on a PC, every animation can be checked as an image before it reaches the real LEDs.

# 3. Interface contract: inputs and outputs

M8 has one software interface (to M7), one build-time input (the city plan) and several physical interfaces (to M7's sensor and motors). All are fixed here.

## 3.1 Software inputs and outputs

| Direction | Item | From / to | When | Content |
|---|---|---|---|---|
| In | `BoardConfig` | M7 | At connect and on change | 8 player tones, brightness 0–1, current limit in mA, animation settings |

| Direction | Item | From / to | When | Content |
|---|---|---|---|---|
| In | `BoardState` | M7 | Up to 60 Hz (normally 30 Hz) | Game phase, players, delivery points, obstacles, earthquake (same fields as the STATE message in the M7 proposal) |
| In | `LinkStatus` | M7 | On change | `CONNECTED`, `SEARCHING` or `LOST` |
| In | City plan | Shared `city_plan.json` | Build time | Road grid and store positions; turned into the LED map (Section 4) |
| Out | LED light | The board | 60 fps | What the players see |
| Out | Frame time, current estimate | M7 | Every frame | Reported to the admin panel in M7's STATUS and PONG |

```
struct PlayerDot   { uint8_t id, team, tone, flags; float u, v, headingDeg,
speedMps; };
struct DeliveryDot { uint8_t playerId; float u, v; uint8_t state; };
struct Obstacle    { uint8_t segment, type, lane; };
struct Quake       { bool active; float u, v; uint8_t magnitude; uint16_t
elapsedMs; };

struct BoardState {
  uint32_t seq, serverMs; uint8_t phase; uint16_t timeLeftS;
  uint8_t nPlayers;    PlayerDot   players[8];
  uint8_t nDeliveries; DeliveryDot deliveries[8];
  uint8_t nObstacles;  Obstacle    obstacles[60];
  Quake quake;
};

// Implemented by M8, called by M7 (see the M7 proposal, Section 3.2)
void ledEngine_applyConfig(const BoardConfig& cfg);
void ledEngine_applyState(const BoardState& st);
void ledEngine_setLinkStatus(LinkStatus s);
void ledEngine_render(uint32_t nowMs, Rgb* out);   // pure: same state + time
= same frame
```

`ledEngine_render` writes into a plain colour array. A separate small driver sends that array to the strips; on a PC the same array is saved as an image. This split is what makes the engine testable without hardware.

### 3.2 Physical interfaces

| Interface | With | Specification |
|---|---|---|
| LED data lines | ESP32 (M7 pin map) | 5 lines, GPIO 4, 16, 17, 18, 23, through a 3.3 V to 5 V level shifter |
| IMU mount | M7 | Centre of the underside, screwed, axes along the roads, x to the east |
| Motor mounts | M7 | 4 printed clamps near the corners of the underside, 130-size motor with weight guard |
| Feet | M7 (hit area method) | 4 soft rubber feet, about 20 mm, so the board can tilt slightly |
| Power input | Group | One 5 V 6 A supply for LEDs and ESP32; a separate 5 V 3 A input for motors |
| Electronics bay | M7 | Removable cover under the board, USB port reachable from the side |

## 4. Board geometry and LED map

The road centre lines form a 6 × 6 grid with a 100 mm pitch, inside a 50 mm frame. At 60 LEDs per metre (16.7 mm between LEDs) every road segment gets exactly 6 LED steps, so the LED grid and the city grid line up with no rounding.

| Measure | Value |
|---|---|
| Base | 600 × 600 mm |
| Frame (edge to outer road centre) | 50 mm on each side |
| Road grid | 6 east–west and 6 north–south lines, 500 mm span, 100 mm pitch |
| City blocks | 5 × 5, about 85 × 85 mm of free space each |
| East–west strips | 6 strips of 31 LEDs, one LED on every intersection |
| North–south pieces | 30 pieces of 5 LEDs, between intersections |

| Measure | Value |
|---|---|
| Store LEDs | 4 stores × 2 LEDs, inside the corner blocks |
| Total | 186 + 150 + 8 = 344 LEDs |

### 4.1 Wiring channels

Short chains limit the damage of one broken LED and keep each data line fast. Pieces are joined under the board through 3 mm holes at the strip ends.

| Channel | ESP32 pin | Covers | LEDs | Data time per frame |
|---|---|---|---|---|
| CH0 | GPIO 4 | East–west rows 0–2, snake order | 93 | 2.8 ms |
| CH1 | GPIO 16 | East–west rows 3–5, snake order | 93 | 2.8 ms |
| CH2 | GPIO 17 | North–south columns 0–2 | 75 | 2.3 ms |
| CH3 | GPIO 18 | North–south columns 3–5 | 75 | 2.3 ms |
| CH4 | GPIO 23 | Store LEDs | 8 | 0.3 ms |

### 4.2 LED map

A script `make_led_map.py` reads `city_plan.json` and a small strip-layout file (where each strip starts, its direction and LED count) and writes two outputs:

- `led_map.h`, a table for the firmware: for every LED its channel, index, road segment and position `u, v`.
- `led_map.json`, the same table for the PC tools (M7's `board_emu.py` draws the board from it).

If a strip is glued a few millimetres off, only its line in the layout file changes. To place a player, the engine finds the nearest road line, converts the position along it to a fractional LED index, and splits brightness between the two nearest LEDs (sub-pixel drawing), so dots slide smoothly instead of jumping 16.7 mm at a time.

## 5. Equipment, parts list and alternatives

Recommendation: WS2812B strips at 60 LEDs per metre on a black PCB, a 9 mm plywood base, 3D-printed white diffusers over the roads, and a closed 5 V desktop power supply. These are the most common parts in LED projects, so spares and help are easy to find.

## 5.1 LED strip

| Option | Strengths | Weaknesses | Verdict |
|---|---|---|---|
| **WS2812B, 60/m, black PCB, not waterproof** | Very common; large pads, easy to solder; 344 LEDs is easy on power and data; exactly 6 LEDs per segment | Coarser steps (solved by sub-pixel drawing) | Recommended |
| WS2812B, 144/m | Very smooth motion | About 880 LEDs: 2.5× the current and data time; tiny pads on 150 joints | Rejected |
| SK6812 RGBW, 60/m | Real white channel for puddles, accidents and "Find me" | 33% more data; a few library defaults assume RGB | Good alternative |
| WS2815, 12 V | Backup data line: one dead LED does not break the chain; less voltage drop | 4 wires per joint; 12 V supply and a 5 V converter for the ESP32 | Alternative if dead LEDs become a problem in tests |
| APA102 / SK9822 | Clocked data, no timing trouble at all | Two data wires; more expensive and harder to find | Rejected |

## 5.2 Parts list

| Part | Qty | Note |
|---|---|---|
| WS2812B strip 60/m, 5 m reel | 2 | 6.5 m needed including cut waste; the rest are spares |
| Single WS2812B modules | 10 | 8 for the stores, 2 spare |
| SN74AHCT245 level shifter | 2 | 3.3 V to 5 V for the 5 data lines; 1 spare |
| 330 Ω resistor, 1000 µF 10 V capacitor | 6, 2 | Data line protection; supply smoothing |

| Part | Qty | Note |
|---|---|---|
| 5 V 6 A closed desktop supply (for example Mean Well GST60A05) | 1 | LEDs and ESP32; software limit 4 A |
| 5 V 3 A supply | 1 | Motors only (M7) |
| Panel DC jack, fuse holder with 5 A fuse, power switch | 1 each | Safe power entry |
| Wire: 18 AWG silicone, 22 AWG, 26 AWG | 3 m, 5 m, 10 m | Power bus, strip power, data |
| 9 mm plywood, 600 × 600 mm | 2 | One board, one for tests |
| PLA and PETG filament | 2 kg, 1 kg | Buildings (PLA); diffusers and stores (white or clear PETG) |
| Rubber feet, about 20 mm | 4 | Soft enough for the hit area method |
| M3 screws, nuts, heat-set inserts | Set | Buildings and mounts |

### 5.3 Tools

Soldering station, multimeter, a USB or clamp current meter, the shared 8-channel logic analyser, and access to a 3D printer (the department lab or a member's printer). A laser cutter would make the road diffusers faster but is not required.

## 6. Electrical design

The LED system is limited to 4 A at 5 V in software, on a 6 A supply. At full white, 344 LEDs could draw about 20 A, but the game never needs that: a busy game frame (8 bikes, 8 delivery points, 10 blocked roads) is estimated at 1–1.5 A, plus about 0.3 A that the LEDs draw even when dark.

### 6.1 Power

| Rule | Value |
|---|---|
| Software current limit | 4 A; the whole frame is dimmed if the estimate is higher |
| Current estimate per frame | Sum of (R + G + B) / 255 × 20 mA, plus 1 mA per LED |

| Rule | Value |
|---|---|
| Power bus | 18 AWG ring under the board; every east–west strip fed at both ends, every north–south column at top and bottom |
| Voltage at the farthest LED at 4 A | ≥ 4.6 V (checked in calibration) |
| Fuse | 5 A on the LED supply, 3 A on the motor supply |
| Supply smoothing | 1000 µF capacitor at the power entry |
| ESP32 supply | From the 5 V bus through a Schottky diode, so a connected USB cable cannot push power back into the bus |
| Ground | LED, motor and ESP32 grounds joined at one point near the power entry |

### 6.2 Data lines

The ESP32 works at 3.3 V, but WS2812B LEDs at 5 V need at least 3.5 V for a reliable "1". A SN74AHCT245 chip raises all 5 data lines to 5 V. Each line has a 330 Ω resistor right before its first LED to damp reflections, and the data wire runs next to a ground wire. The first LED of each channel sits within 50 cm of the ESP32.

### 6.3 Safety

No mains (230 V) wiring is done by the team: both supplies are closed desktop adapters. Any wire that carries more than 1 A is silicone-insulated 18 AWG. The underside has a cover, so no solder joint can be touched when the board is lifted.

## 7. Animation engine

Each frame is built by drawing five layers from bottom to top, so a more important light always covers a less important one: a motorcycle on a blocked road is still visible, and "Find me" is never hidden. Every effect is a pure function of the state and the time, so the same input always gives the same frame.

### 7.1 Frame pipeline (60 fps)

1. Take the newest `BoardState` from the double buffer.
2. Move the motorcycle dots smoothly: draw them one state update (33 ms) behind, interpolating between the last two states. This adds 33 ms of delay but removes jumps. If a state is missing, continue the last motion for at most 100 ms.

3. Draw the layers (7.2).
4. Apply gamma correction (2.2), global brightness and the current limit.
5. Hand the colour array to the LED driver, which sends all 5 channels in parallel in the background.

## 7.2 Layers

| Order | Layer | Look |
|---|---|---|
| 1 (bottom) | Roads | Off, or a very dim warm white (admin setting) so the roads can be seen |
| 2 | Obstacles | Whole segment, pattern by type (7.3) |
| 3 | Delivery points | 2 LEDs blinking at 1 Hz in the player's tone; a short bright flash when delivered |
| 4 | Motorcycles | Head at full brightness in the player's tone, 2-LED fading tail behind it; a fallen player pulses slowly |
| 5 (top) | Overlays | "Find me" white flash for 1 s; earthquake shock wave |

Stores (CH4) are drawn apart from the roads: each store LED shows the tone of one team player; with one player both LEDs show the same tone. A printed wall inside the store separates the two LEDs, so the tones do not mix.

## 7.3 Effects

| Effect | Pattern on the LEDs |
|---|---|
| Heavy traffic | Dim red (about 25%) with a slow wave moving along the road, 2 s period |
| Road work | Bright steady red (about 80%) |
| Accident | Red and white alternating at 2 Hz |
| Water puddle | Cool white shimmer: random soft twinkle at 30–60% brightness |
| Shock wave | A ring that grows from the centre at about 250 mm/s on the board, 2–3 LEDs wide; brightness and duration grow with the magnitude |

| Effect | Pattern on the LEDs |
|---|---|
| Game phases | Lobby: team colours slowly rotating on the outer road; countdown: outer road fills in 3 steps; finished: board fills with the winning team's colour |
| Link lost | All roads off; store LEDs breathe in amber until the link returns |

Team colours are far from red in hue, as required by the project proposal. The exact LED values of the 8 tones are tuned on the real board (Section 9) and sent by the server in `BoardConfig`, so the 3D city and the board use the same tone names.

### 7.4 Demo mode

Without any server, a button press (or the USB command `demo`) starts a built-in scenario: 8 bikes driving loops, every obstacle type, delivery points and an earthquake every 20 s. This shows that M8 works alone and is also used for the long burn-in test.

## 8. Mechanical build

The board is a stiff plywood base with LEDs on top under printed covers, all wiring underneath, and buildings fixed so that a hand slap cannot move them. It is built in seven steps, and each step is tested before the next one closes it.

1. **Base.** Cut 9 mm plywood to 600 × 600 mm, paint it matte dark grey, and drill 3 mm holes at every strip end, plus holes for the IMU and motor mounts. A drilling template is printed from the LED map, so holes match the strips.
2. **Strip test.** Light every cut strip piece in red, green, blue and white on the bench before gluing. A dead LED found now costs 2 minutes; found after gluing it costs an hour.
3. **Strips.** Glue the strips along the road lines (their own tape plus a drop of hot glue at each end). Solder the joints under the board.
4. **Road covers.** Clip 3D-printed white PETG covers (14 mm wide, 1 mm thick top) over every strip. They spread each LED into a soft dot and protect the LEDs from hits.
5. **Wiring bay.** Mount the ESP32, level shifter, fuse and power entry on a printed tray under the board, with a removable cover and the USB port reachable from the side.
6. **Sensor and motors.** Screw the IMU at the centre and the four motor clamps near the corners (positions agreed with M7). Add the four soft feet.
7. **Buildings.** Print the buildings and fix them with an M3 screw or two pins each. Tall buildings in the centre, houses and trees outside, translucent store buildings with an inner wall in the four corner blocks.

### 8.1 Buildings

The buildings come from the same low-poly models as the 3D city (M5), exported to STL at board scale (1 m in the game = 1 mm on the board). This keeps the physical and 3D cities the same, which the project proposal requires.

| Area | Building height on the board | Count (estimate) | Print time (estimate) |
|---|---|---|---|
| Centre (9 blocks) | 50–100 mm | about 18 | about 25 h |
| Middle and outer ring (12 blocks) | 20–40 mm, plus trees | about 30 | about 20 h |
| Corner stores (4 blocks) | about 50 mm, translucent | 4 | about 6 h |

Prints use 0.28 mm layers and 10% infill to save time. With one printer the buildings take about 2 weeks of evening printing, so printing starts in week 2.

## 9. Calibration procedures

Six calibrations make the board match the city plan and keep it safe. All test patterns are built into the firmware and started with a USB command, so they can be repeated on demo day.

| Step | What | How | Saved result | Accept when |
|---|---|---|---|---|
| L1 | LED order | Pattern lights LEDs one by one per channel, then each road segment in its own colour | Fixes in the strip-layout file | Every segment lights at the right place; no LED missing |
| L2 | Position | Pattern lights every intersection LED; compare with a printed grid placed on the board | Offsets in the strip-layout file | Every intersection LED within 5 mm of its grid point |

| Step | What | How | Saved result | Accept when |
|---|---|---|---|---|
| L3 | Colours | Show the 8 player tones and the 4 obstacle patterns side by side under classroom light; 3 people name each one | Final RGB values of the tones | Every tone named correctly by all 3 people; no tone mistaken for red |
| L4 | Brightness and current | Full-white test frame with the limit on; measure the current | Confirmed current factor in the estimate | Measured current ⩽ 4.2 A with a 4 A limit |
| L5 | Voltage drop | Same frame; measure the voltage at the farthest LED of each channel | — | ⩾ 4.6 V everywhere |
| L6 | Heat and timing | 30 minutes at the limit; logic analyser on one data line | — | Strips and supply below 50 °C; frame time jitter < 2 ms |

Step L3 matters most for gameplay: players navigate only by the board, so if two tones look alike, a player follows the wrong delivery point. Tones are tuned on the LEDs, not on a screen, because LEDs show colour very differently from a monitor.

# 10. Standalone testing

M8 is tested in two halves that need nothing from other modules: the animation engine on a PC, and the hardware with the built-in demo mode.

## 10.1 Software on the PC

The engine builds as a normal PC program (PlatformIO native build). A test feeds it a `BoardState` and a time, and saves the frame as a PNG image of the board drawn from `led_map.json`. Saved "golden" frames are compared pixel by pixel on every change.

| ID | Test | Pass criterion |
|---|---|---|
| S1 | Every intersection and segment midpoint maps to the expected LED | All 36 + 60 positions correct |

| ID | Test | Pass criterion |
|---|---|---|
| S2 | Positions slightly off the road line (up to 3 m in the game) still snap to the right road | Correct road in 10,000 random cases |
| S3 | Layer order: bike over obstacle, "Find me" over everything | Golden frames match |
| S4 | Each effect at fixed times | Golden frames match |
| S5 | Current limit on random frames | Estimated current never above the limit |
| S6 | Missing or late states | No jump larger than 1 LED; bikes stop after 100 ms of extrapolation |
| S7 | Render time on the ESP32 for the busiest frame | ⩽ 6 ms |

### 10.2 Hardware with demo mode

| ID | Test | Pass criterion |
|---|---|---|
| H1 | Strip test before gluing (build step 2) | Every LED shows red, green, blue and white |
| H2 | Calibrations L1–L6 | All accepted |
| H3 | Burn-in: demo mode for 2 hours | No restart, no flicker, temperatures as in L6 |
| H4 | Hit test: 20 hand slaps of growing force on different areas | No building, cover or strip moves; no LED fails |
| H5 | Transport test: carry the board between rooms twice | Still passes L1 and H1 afterwards |

## 11. Integration with other modules

M8 integrates first with M7 on the same chip, then with the server through M7, and last with the 3D client for colour and position matching.

| Order | Module | What is exchanged | Integration test | Pass criterion |
|---|---|---|---|---|
| 1 | M7 firmware | `BoardState`, `BoardConfig`, link status; frame time and current back | `server_emu.py` scenarios drive the real board | Board frames match the PC golden frames for the same scenario |
| 2 | M7 sensor and motors | Mounts, feet, cables | M7 calibrations C3–C5 on the finished board | M7 targets met; no part loosens during motor runs |
| 3 | M1 via M7 | Real game state | Alignment scenario: one player visits every intersection | LED dot and 3D bike on the same intersection every time |
| 4 | M5 Player Client | Tone names and colours, building models | Compare phone and board side by side in a live game | Each player names their own tone on the board within 3 s |
| 5 | M6 Admin Panel | Obstacles and earthquakes from the panel | Add every obstacle type and start an earthquake | Right pattern on the board; board delay graph below 100 ms |

The shared `city_plan.json` decides where every LED is. If its version changes, `make_led_map.py` is run again and S1, S2 and L1, L2 are repeated before the next integration test.

## 12. Risks, work plan and open decisions

| Risk | Likelihood | Impact | Mitigation |
|---|---|---|---|
| Dead LED or broken joint after assembly | Medium | High | Strip test before gluing; short chains (≤ 93 LEDs); spare strip; joints reachable from below |
| Board damaged by earthquake hits | Medium | High | Road covers, screwed buildings, hit test H4 |
| Player tones confused with each other or with red | Medium | High | Calibration L3 with real people; SK6812 RGBW as backup strip |

| Risk | Likelihood | Impact | Mitigation |
|---|---|---|---|
| Voltage drop makes far LEDs change colour | Low | Medium | Power ring fed at both ends; L5 check |
| 3D printing takes longer than planned | High | Medium | Start in week 2; simple low-poly models; the game works with fewer buildings |
| Board plan and 3D city drift apart | Medium | High | Both made from the same city plan and models; alignment test |
| Plywood bends and strips come loose | Low | Medium | 9 mm plywood, cross braces under the board |

## Work plan

Week numbers count from the approval of this proposal.

1. **Weeks 1–2:** Order parts. Engine on the PC with golden-frame tests S1–S6. Final building models; printing starts.
2. **Weeks 3–4:** Cut and test strips (H1); base, holes and wiring; first light with demo mode on the ESP32 (S7).
3. **Weeks 5–6:** Road covers, wiring bay, sensor and motor mounts; calibrations L1–L6; integration step 1 with M7.
4. **Weeks 7–8:** Buildings mounted; tests H3–H5; integration steps 2–3.
5. **Weeks 9–10:** Integration steps 4–5, tone tuning in real games, final photos and report data.

## Decisions needed from the group before submission

- [ ] 60 LEDs per metre and the 100 mm grid (sets the board size of the city plan)
- [ ] Stores inside the corner blocks (must match M4 and M5)
- [ ] Scale 1 m in the game = 1 mm on the board (M5 must use the same building models)
- [ ] Access to a 3D printer and, if possible, a laser cutter
- [ ] Budget owner for the parts list